

Углубленное машинное обучение и искусственный интеллект в Python

Патласов Дмитрий Александрович, ПГНИУ
dmitriypatlasov@gmail.com

Введение в генеративное моделирование

Здесь рассматриваются приемы и методы, которые анализируют данные и генерируют совершенно новые экземпляры данных на основе изученных закономерностей из модели.

Вы можете не сразу это заметить, но следующие лица были сгенерированы с помощью генеративной модели, обученной на большом наборе данных с лицами.



На предыдущей лекции основное внимание было уделено методам контролируемого обучения, которые предполагают построение моделей, способных отображать входные данные в соответствующие метки на основе обучающей выборки с размеченными данными.

В отличие от этого, неконтролируемое обучение направлено на анализ и моделирование структуры данных без использования predetermined меток. Основная цель таких методов заключается в выявлении скрытых закономерностей или внутренней структуры распределения данных. К типичным примерам неконтролируемого обучения относятся алгоритмы кластеризации и метод главных компонент (PCA), широко применяемые в классическом машинном обучении.

Supervised vs unsupervised learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn function to map
 $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, etc.

Unsupervised Learning

Data: x

x is data, no labels!

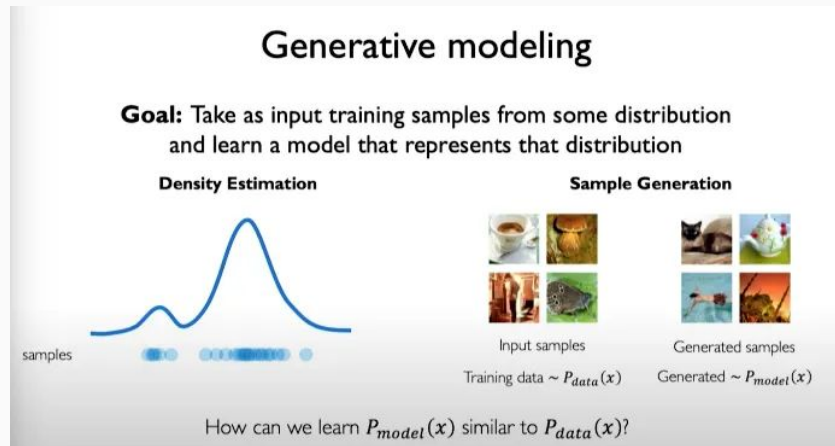
Goal: Learn the *hidden* or *underlying structure* of the data

Examples: Clustering, feature or dimensionality reduction, etc.

Глубокие генеративные модели представляют собой разновидность неконтролируемого обучения. Целью таких моделей является изучение вероятностного распределения, лежащего в основе заданных данных. Особый акцент сделан на два ключевых направления: оценку плотности распределения и генерацию новых образцов данных.

Формально задача заключается в построении параметрической модели, способной аппроксимировать истинное распределение данных, порождающее выборку. Это достигается путём оптимизации функции правдоподобия или других связанных критериев, направленных на согласование модельного и эмпирического распределений.

Данная задача представляет как теоретический, так и практический интерес, поскольку зачастую данные имеют высокую размерность — например, изображения, тексты или аудиосигналы. В таких случаях истинное распределение не поддаётся описанию с помощью простых аналитических функций, таких как нормальное или равномерное распределение. Для моделирования подобных сложных распределений особенно эффективно применяются глубокие нейронные сети, обладающие способностью аппроксимировать нелинейные функциональные зависимости и выявлять высокоуровневые представления в данных.



Генеративную модель можно использовать не только для аппроксимации распределения скрытых (латентных) факторов, определяющих структуру данных, но и для синтеза нового, сбалансированного и репрезентативного набора данных. Такой подход позволяет устранять систематические смещения, присутствующие в исходных данных, и формировать обучающие выборки, которые более адекватно отражают разнообразие реальных сценариев. В результате становится возможным обучение моделей машинного обучения, обладающих повышенной справедливостью и меньшей предвзятостью по отношению к различным группам, определяемым базовыми признаками.

Why generative models? Debiasing

Capable of uncovering **underlying features** in a dataset



Homogeneous skin color, pose

vs



Diverse skin color, pose, illumination

How can we use this information to create fair and representative datasets?

Генеративные модели также можно использовать, например, в контексте беспилотного вождения, чтобы обнаружить, когда пешеходы переходят дорогу или другие выбросы в данных.

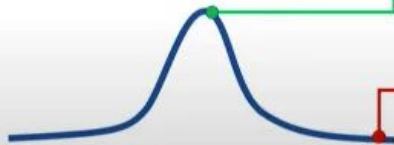
Why generative models? Outlier detection

- **Problem:** How can we detect when we encounter something new or rare?
- **Strategy:** Leverage generative models, detect outliers in the distribution
- Use outliers during training to improve even more!

95% of Driving Data:
(1) sunny, (2) highway, (3) straight road



Detect outliers to avoid unpredictable behavior when training



Edge Cases



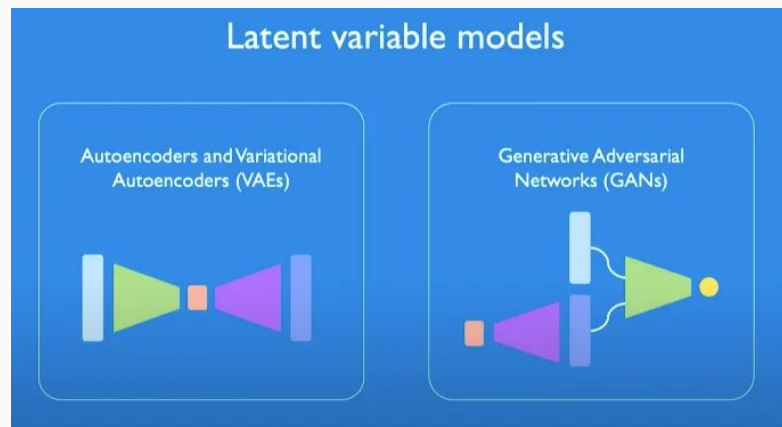
Harsh Weather



Pedestrians

Модели скрытых переменных

- autoencoders and variational autoencoders
- generative adversarial networks (GANs)



Что такое скрытые переменные?

Скрытые (или латентные) переменные — это не наблюдаемые напрямую факторы, которые, как предполагается, лежат в основе наблюдаемых данных и определяют их структуру. Они представляют собой абстрактные, но содержательные характеристики, которые могут объяснять изменчивость и корреляции между наблюдаемыми переменными.

Для иллюстрации можно привести аналогию с пещерой Платона: люди, находящиеся в пещере и видящие только тени на её стене, воспринимают эти тени как наблюдаемые данные. Однако истинные объекты, отбрасывающие эти тени, остаются вне поля их зрения — они невидимы, но именно они порождают наблюдаемые эффекты. Эти недоступные для непосредственного наблюдения объекты служат метафорой скрытых переменных: причинных факторов, которые мы не можем измерить напрямую, но которые формируют структуру доступных нам данных.

Цель генеративного моделирования

Основная задача генеративного моделирования заключается в том, чтобы построить модель, способную восстановить вероятностное распределение наблюдаемых данных, при этом выявляя и используя внутренние, скрытые представления, которые управляют этим распределением. Другими словами, речь идёт о реконструкции потенциальных истинных причинных факторов (скрытых переменных), которые могли бы объяснить наблюдавшиеся данные.

Формально это сводится к построению совместного распределения $p(x, z)$, где x — наблюдаемые данные, а z — соответствующие им скрытые переменные. Задача обучения состоит в том, чтобы, имея лишь выборку из маргинального распределения $p(x)$, научиться восстанавливать как параметры этого распределения, так и значения латентных переменных z , которые наиболее вероятно породили конкретный образец x .

Таким образом, генеративное моделирование стремится раскрыть скрытую причинно-следственную структуру данных и использовать её для генерации новых примеров, оценки плотности, анализа представлений и других задач.

What is a latent variable?



Can we learn the **true explanatory factors**, e.g. latent variables, from only observed data?

Autoencoders

В случае автокодировщиков (autoencoders) задача обучения нейронной сети заключается в том, чтобы извлечь полезное представление входных данных в условиях отсутствия явных меток для латентных переменных z . Это делает автокодировщик методом **неконтролируемого обучения**, поскольку он обучается без использования разметки.

Как работает автокодировщик?

Автокодировщик состоит из двух основных частей:

1. **Энкодер** $q(z|x)$: отображает входные данные $x \in \mathbb{R}^D$ в латентное представление $z \in \mathbb{R}^d$, где обычно $d \ll D$.
2. **Декодер** $p(x|z)$: восстанавливает (реконструирует) исходный входной сигнал x на основе этого латентного представления z .

Цель обучения — минимизировать **функцию потерь реконструкции**, которая измеряет, насколько хорошо декодер может восстановить исходный входной образец из его сжатого латентного представления:

$$\mathcal{L}(x) = \|x - p(q(x))\|^2$$

Таким образом, **цель обучения** формулируется как минимизация ошибки между оригиналом и реконструкцией, что позволяет модели самой "понять", какие аспекты данных важны для сохранения в латентном пространстве.

Autoencoders: background

Unsupervised approach for learning a **lower-dimensional** feature representation from unlabeled training data



"Encoder" learns mapping from the data, x , to a low-dimensional latent space, z

Действительно, в случае автокодировщиков обучение модели происходит без использования явных меток — это пример **неконтролируемого обучения**, где входные данные сами служат основой для обучения.

Формализация подхода

Пусть $x \in \mathbb{R}^D$ — входной образец (например, изображение), а $z \in \mathbb{R}^d$ — его представление в низкоразмерном латентном пространстве, полученное с помощью **энкодера** $z = q(x)$. Затем **декодер** $p(z)$ преобразует это представление обратно в пространство признаков исходного входа, порождая реконструированный выход $\hat{x} = p(q(x))$.

Цель состоит в том, чтобы обучить модель так, чтобы $\hat{x}$ как можно ближе соответствовал оригиналу x . Для этого используется функция потерь, измеряющая расхождение между x и $\hat{x}$. Наиболее распространённым выбором является **среднеквадратичная ошибка (MSE)**:

$$\mathcal{L}_{\text{reconstruction}}(x, \hat{x}) = \|x - \hat{x}\|_2^2 = \sum_{i=1}^D (x_i - \hat{x}_i)^2,$$

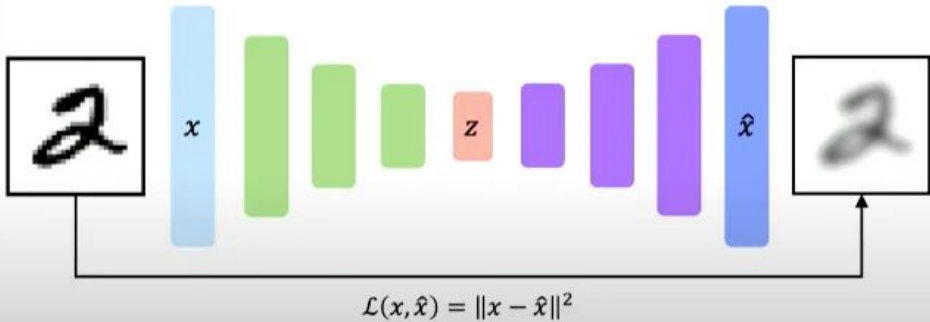
где суммирование ведётся по всем элементам входного вектора (например, пикселям изображения).

Эта функция потерь вычисляется попиксельно (или поэлементно) и служит мерой качества реконструкции. Она не требует внешних меток, поскольку использует сам входной сигнал как целевой выход.

Autoencoders: background

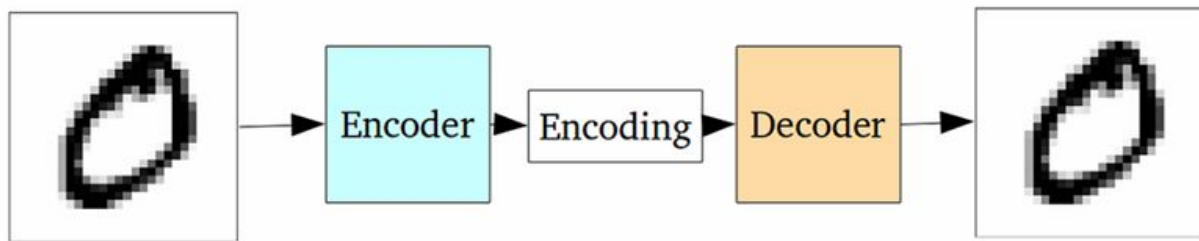
How can we learn this latent space?

Train the model to use these features to **reconstruct the original data**



Автоэнкодер *

- Автоэнкодеры появились в начале 90-ых годов. Это нелинейное обобщение метода главных компонент.
- Энкодер генерирует закодированные данные таким образом, чтобы они были пригодны для восстановления входных данных



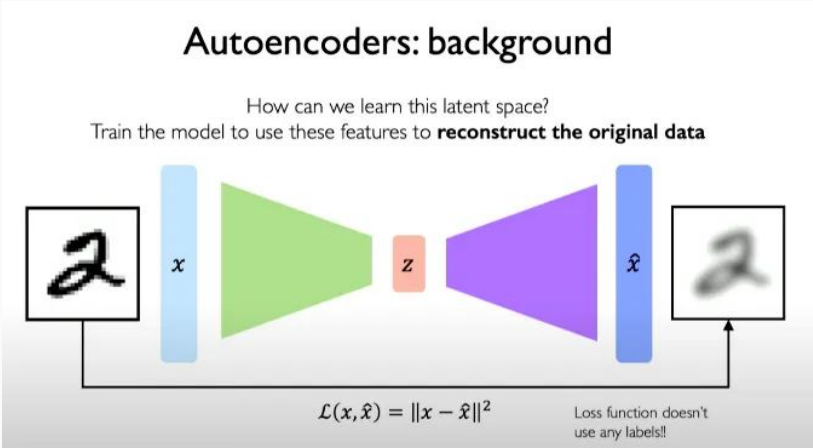
- Функция потерь выбирается как среднеквадратичная ошибка или как кросс-энтропия между входными и выходными данными
- Она не позволяет нейросети создавать выходные данные, сильно отличающиеся от входных

В упрощенном виде архитектура выглядит следующим образом: мы пытаемся охватить те переменные, которые могут иметь смысл, не требуя каких-либо меток для данных.

Насколько хороши реконструкции, зависит от степени сжатия. Чем меньше скрытое пространство, тем меньше скрытых переменных мы должны попытаться узнать, тем хуже качество реконструкции, которое мы получим.

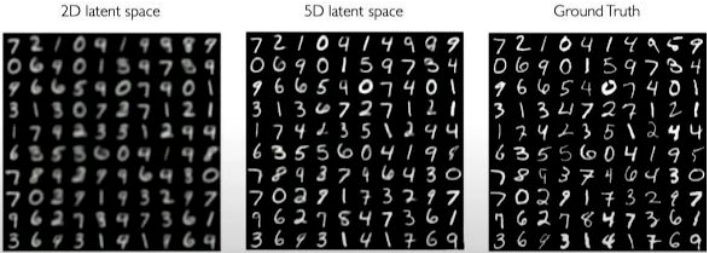
Компромисс между сжатием и качеством реконструкции

- Узкое место (bottleneck) — это размерность латентного представления z . Оно определяет, сколько информации может быть передано от энкодера к декодеру.
- Если $\dim(z)$ очень мало (сильное сжатие), модель вынуждена выбирать, какие аспекты данных важны для сохранения, чтобы минимизировать ошибку реконструкции.
- В результате:
 - При **низкой размерности z** : реконструкция будет грубой, могут теряться детали, особенно если данные сложные (например, изображения высокого разрешения).
 - При **высокой размерности z** : модель может почти идеально восстановить вход, но тогда латентное представление теряет свойство компактности и способность обобщать.



Dimensionality of latent space → reconstruction quality

Autoencoding is a form of compression!
Smaller latent space will force a larger training bottleneck



Автокодировщики представляют собой эффективный метод **обучения представлениям** в режиме **неконтролируемого обучения**, где задача заключается в извлечении сжатых и информативных представлений входных данных без использования явных меток.

Определение и структура

Автокодировщик — это нейронная сеть, состоящая из двух основных компонентов:

1. **Энкодер (Encoder)** : отображает входные данные $x \in \mathbb{R}^D$ в скрытое (латентное) представление $z \in \mathbb{R}^d$, где $d < D$. Это достигается с помощью функции $z = q(x)$.
2. **Декодер (Decoder)** : восстанавливает данные из латентного представления z , порождая реконструированный выход $\hat{x} = p(z)$, который должен быть как можно ближе к исходному x .

Таким образом, модель учится **автоматически кодировать** и затем **декодировать** входные данные, отсюда и название — *автокодировщик* (autoencoder).

Autoencoders for representation learning

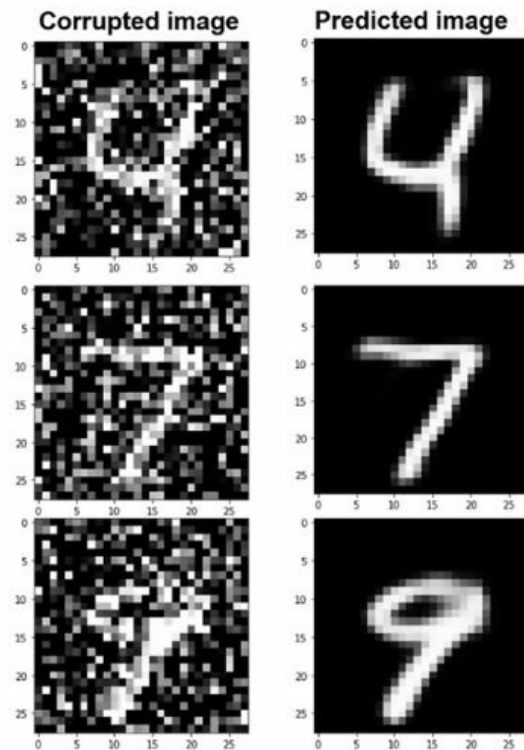
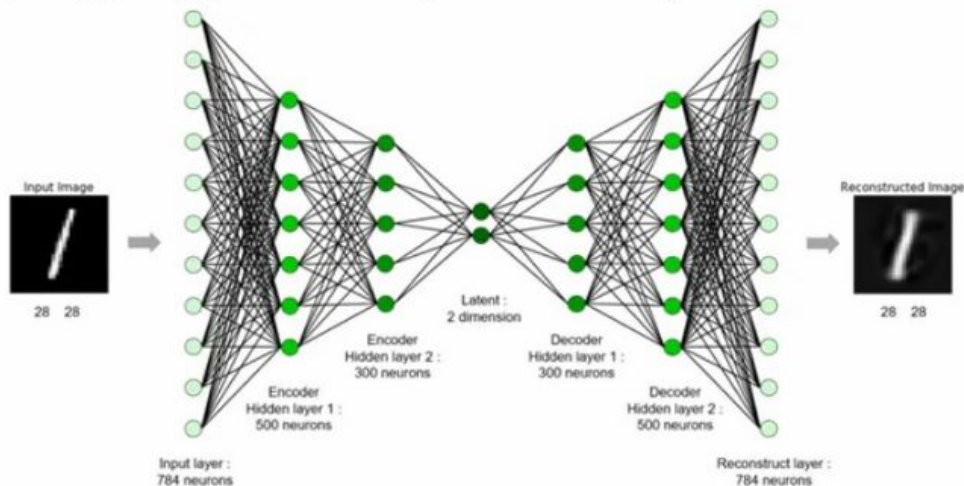
Bottleneck hidden layer forces network to learn a compressed latent representation

Reconstruction loss forces the latent representation to capture (or encode) as much "information" about the data as possible

Autoencoding = **Automatically encoding** data; "Auto" = **self**-encoding

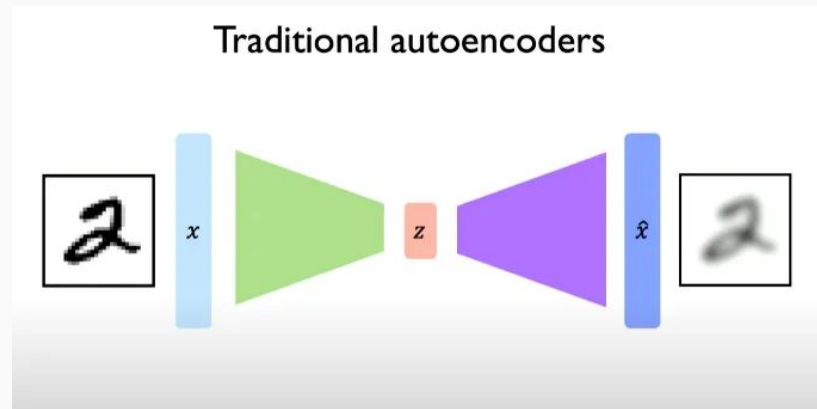
Denosing Autoencoder

- Обучаем модель на восстановлении изображения
- Используем для удаления шума из изображения



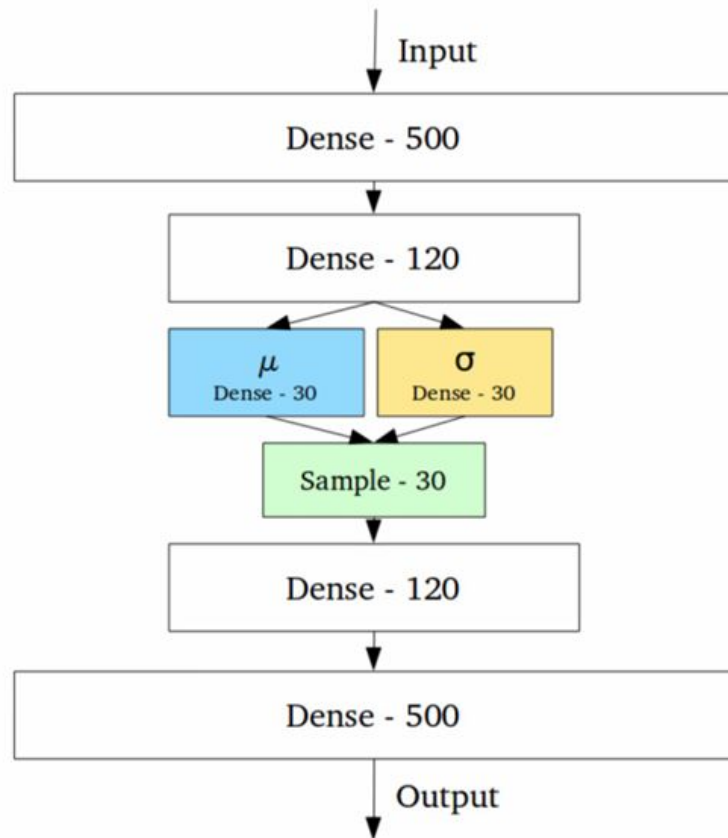
Variational Autoencoders (VAEs)

С традиционным автокодировщиком мы всегда будем получать один и тот же вывод, независимо от того, сколько раз мы передадим один и тот же ввод; один ввод, один вывод. Кодирование и декодирование, которые мы изучаем, являются детерминированными, как только сеть полностью обучена.



Вариационный автоэнкодер *

- Вариационный автоэнкодер (Variational Autoencoder – VAE) имеет отличие от стандартного автоэнкодера.
- Их скрытое пространство по построению является непрерывным, позволяя выполнять случайные преобразования и интерполяцию.
- Непрерывность скрытого пространства достигается неожиданным способом: энкодер выдаёт не один вектор размера n , а два вектора размера n – вектор средних значений μ и вектор стандартных отклонений σ .



Вариационный автокодировщик (Variational Autoencoder , VAE) представляет собой вероятностное обобщение классического автокодировщика, которое позволяет не только извлекать латентные представления данных, но и генерировать новые образцы , сохраняя при этом гладкость и полноту скрытого пространства.

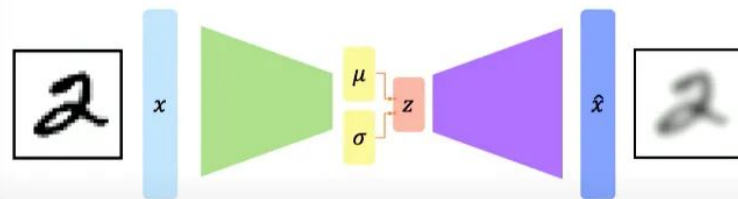
Основная идея VAE

Цель VAE — обучить модель, способную:

- Кодировать входные данные x в распределение над латентными переменными z .
- Декодировать z обратно в пространство данных, восстанавливая x .

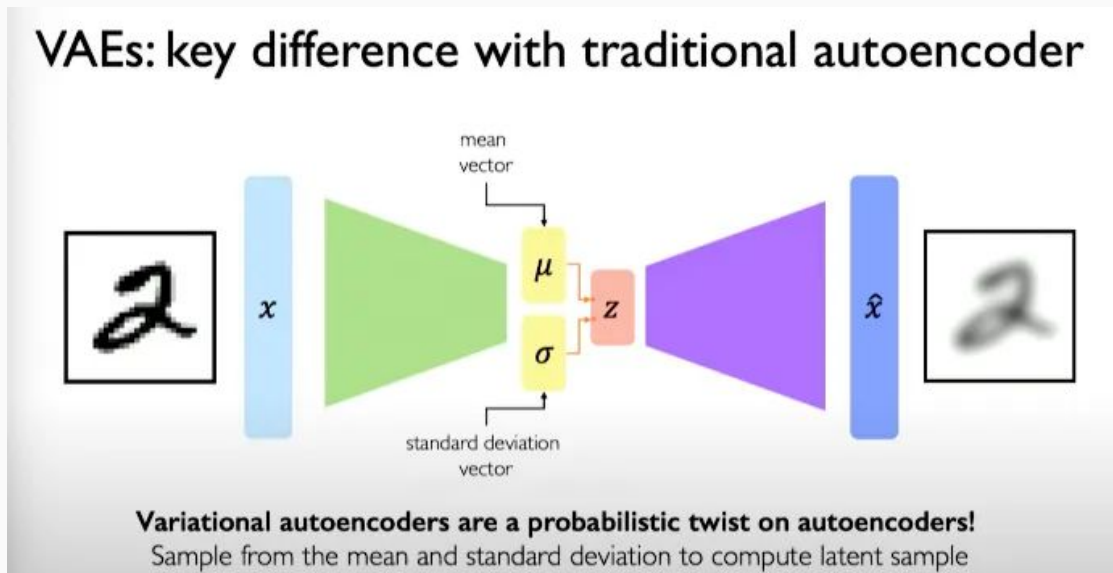
Обеспечивать гладкое и непрерывное латентное пространство, что делает возможным генерацию новых, ранее невидимых образцов.

VAEs: key difference with traditional autoencoder



С помощью VAE мы разбиваем скрытое пространство z на среднее значение и стандартное отклонение. Цель кодировщика — вывести средний вектор и вектор стандартного отклонения, которые соответствуют распределениям этих скрытых переменных z .

По сути, мы вводим элемент вероятности, некий элемент случайности, который позволит нам теперь генерировать новые данные для построения более осмысленного и более информативного скрытого пространства.



VAE Optimization

В вариационном автокодировщике (VAE) как **энкодер**, так и **декодер** моделируются как **вероятностные отображения**, что делает всю архитектуру вероятностной по своей природе.

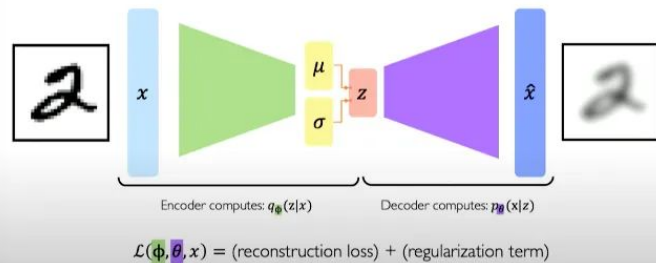
Вероятностная интерпретация кодера и декодера

- **Энкодер** :
Обозначается как $q_{\phi}(z|x)$ — параметризованное нейросетью распределение над латентными переменными z , зависящее от входа x . Цель энкодера — вывести **апостериорное распределение** на z , приближающее истинное $p(z|x)$.
- **Декодер** :
Обозначается как $p_{\theta}(x|z)$ — условное распределение на пространстве данных x , заданное фиксированным значением z . Декодер моделирует, как могут выглядеть данные, если известно некоторое представление в латентном пространстве.

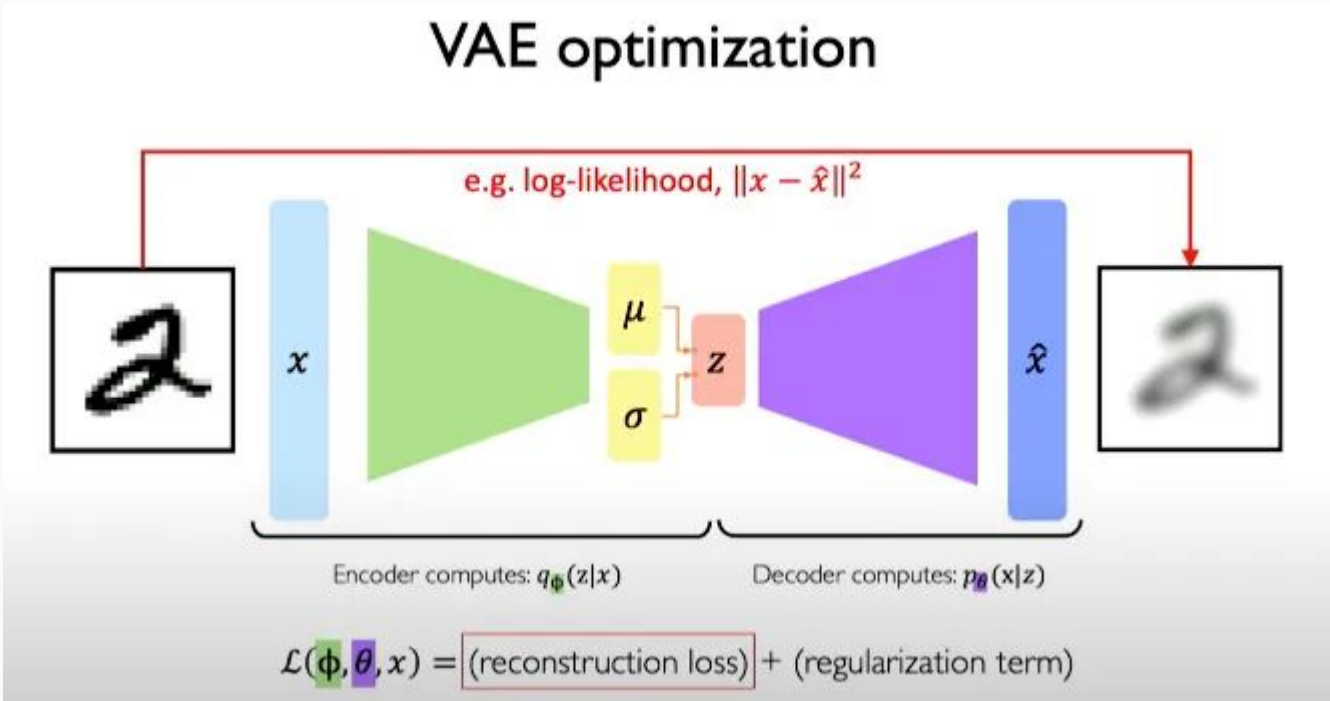
Таким образом, модель опирается на два параметризованных компонента:

- ϕ : параметры энкодера,
- θ : параметры декодера.

VAE optimization



Ф-я потерь при реконструкции следующая:



VAE Loss & regularization

Функция потерь: Evidence Lower Bound (ELBO)

Цель обучения VAE — максимизация нижней границы правдоподобия данных, известной как **ELBO** :

$$\mathcal{L}_{\text{ELBO}}(x; \phi, \theta) = \underbrace{\mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x|z)]}_{\text{Реконструкционная потеря}} - \underbrace{D_{\text{KL}}(q_{\phi}(z|x) \parallel p(z))}_{\text{Регуляризационный член}}$$

Где:

- **Реконструкционная потеря** измеряет, насколько точно декодер может восстановить вход x из выборки z , взятой из распределения $q_{\phi}(z|x)$.
- **Регуляризационный член** (KL-дивергенция) штрафует отклонение апостериорного распределения $q_{\phi}(z|x)$ от заданного априорного распределения $p(z)$, обычно стандартного нормального: $p(z) = \mathcal{N}(0, I)$.

Роль регуляризации

Добавление KL-дивергенции в функцию потерь играет ключевую роль:

- Она **регулирует структуру латентного пространства**, обеспечивая его гладкость и непрерывность.
- Это позволяет брать выборки $z \sim p(z)$ и использовать их для генерации новых образцов x , даже тех, которых не было в обучающей выборке.
- Без этого члена латентное пространство могло бы быть плохо структурированным или разреженным, что затрудняло бы генерацию и интерпретацию.

Generative Adversarial Networks (GANs)

Что, если мы хотим сосредоточиться на создании высококачественных новых образцов в качестве наших выходных данных? Для этого мы можем использовать новый тип генеративной модели, называемой GAN.

Мы не хотим явно моделировать распределение вероятностей данных, как мы делали с VAE. Здесь мы заинтересованы в использовании информации для выборки реалистичных и новых экземпляров данных, которые соответствуют входным распределениям.

Проблема в том, что входные данные невероятно сложны, что делает сложным переход от входных данных к непосредственному созданию новых реалистичных образцов.

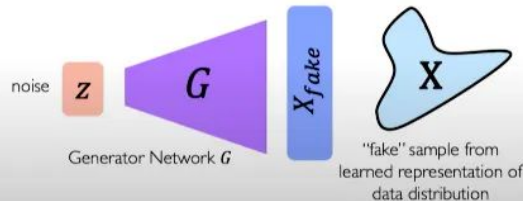
Основная идея GAN заключается в том, чтобы начать с чего-то простого, случайного шума, и использовать нейронные сети для обучения преобразованию из этого очень простого распределения в наше целевое распределение данных, из которого мы затем можем делать выборки.

What if we just want to sample?

Idea: don't explicitly model density, and instead just sample to generate new instances.

Problem: want to sample from complex distribution – can't do this directly!

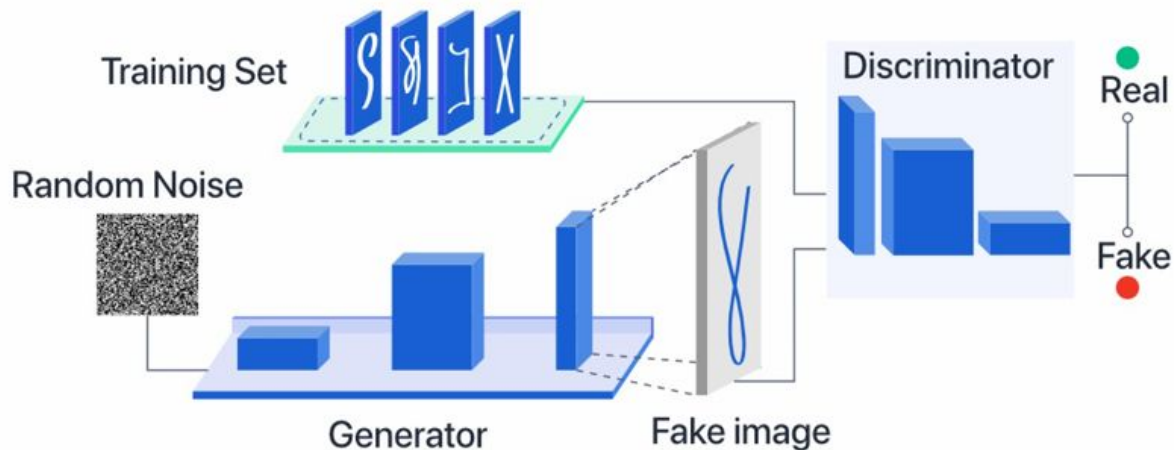
Solution: sample from something simple (e.g., noise), learn a transformation to the data distribution.



GAN *

- Генеративно-сопоставительная сеть (Generative adversarial network)

- Дискриминативная отличает правильные от неправильных



Описал Ян Гудфеллоу в 2014

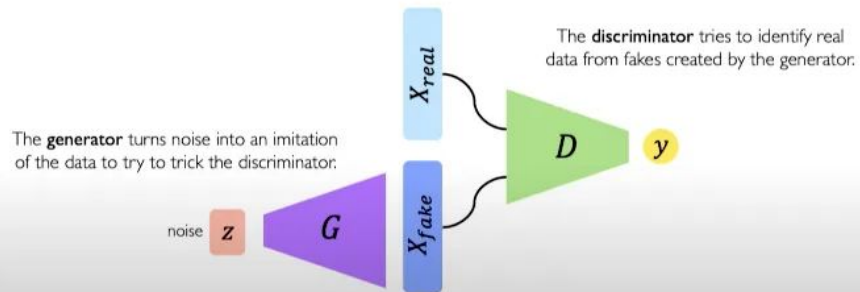
- Генеративная модель создает образы

GAN состоит из двух отдельных нейронных сетей, которые являются противниками. Первый компонент — это сеть-генератор, которая черпает данные из очень простого входного распределения, чисто случайного шума. Затем она использует этот шум для преобразования его в имитацию реальных данных. И наоборот, у нас есть сеть-соперник, дискриминатор, который берет образцы, сгенерированные генератором, полностью поддельные образцы, и предсказывает, являются ли эти образцы реальными или поддельными.

По сути, мы создаем соревнование между этими двумя сетями, заставляя дискриминатор классифицировать реальные и поддельные данные и заставляя генератор выдавать все более и более качественные поддельные данные, чтобы попытаться обмануть дискриминатор.

Generative Adversarial Networks (GANs)

Generative Adversarial Networks (GANs) are a way to make a generative model by having two neural networks compete with each other:



Отличный переход к следующему важному классу генеративных моделей — **генеративным состязательным сетям (GAN, Generative Adversarial Networks)** . В отличие от VAE, которые явно моделируют распределение данных и работают в рамках вероятностного вывода, GAN предлагают совершенно иной подход, основанный на **игровой интерпретации обучения** .

Основная идея GAN

Цель GAN — научиться генерировать новые данные, которые похожи на реальные образцы из обучающего набора, без необходимости явного моделирования плотности распределения $p(x)$. Это делает их особенно мощными для задач, где качество сгенерированных образцов имеет приоритет над точностью оценки вероятностей.

Архитектура GAN

GAN состоит из двух нейронных сетей, которые участвуют в **минимаксной игре** :

1. Генератор $G(z; \theta_g)$:

- Принимает на вход случайный шум z из простого априорного распределения (например, $z \sim \mathcal{N}(0, I)$).
- Преобразует его в синтезируемый образец $x_{\text{fake}} = G(z)$, который должен быть неотличим от реальных данных.
- Цель: "обмануть" дискриминатор, создавая данные, которые он не сможет отличить от настоящих.

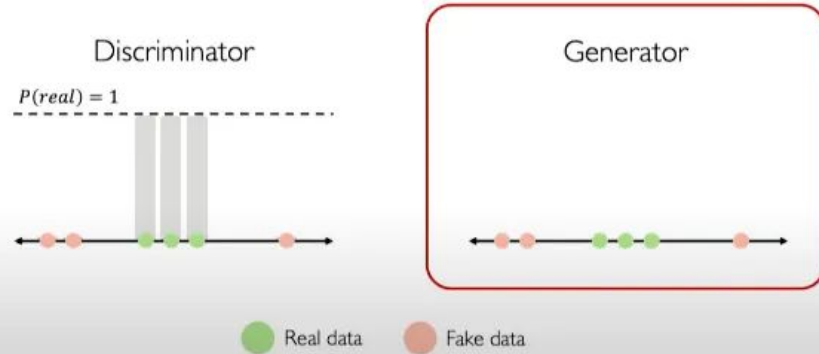
2. Дискриминатор $D(x; \theta_d)$:

- Получает либо реальный образец x_{real} из обучающей выборки, либо сгенерированный x_{fake} .
- Выдает вероятность того, что вход является реальным: $D(x) \in [0, 1]$.
- Цель: корректно классифицировать поданные образцы как реальные или сгенерированные.

Затем цель состоит в том, чтобы скормить генератору некоторые образцы реальных данных и переместить поддельные примеры ближе к реальным данным, все больше приближая их так, чтобы теперь дискриминатор вернулся, получил новые точки и оценил вероятности того, что каждая точка является реальной.

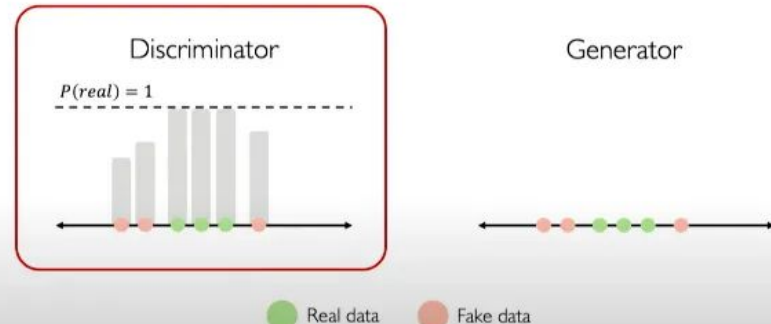
Intuition behind GANs

Generator tries to improve its imitation of the data.



Intuition behind GANs

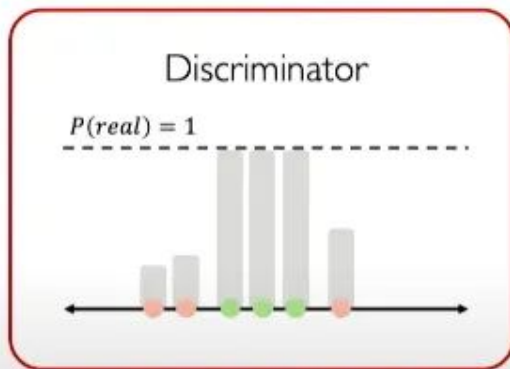
Discriminator tries to predict what's real and what's fake.



Он итеративно учится уменьшать вероятность появления фальшивых объектов

Intuition behind GANs

Discriminator tries to predict what's real and what's fake.



Generator



Real data

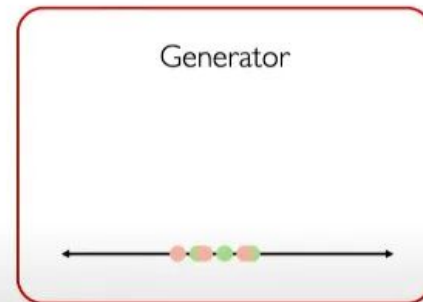
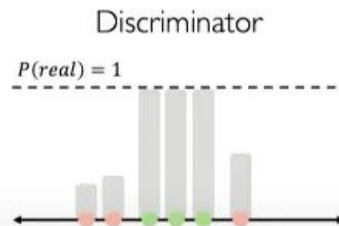


Fake data

Затем мы можем продолжить корректировать вероятности до тех пор, пока в конце концов мы снова не вернемся к генератору, и в последний раз генератор начнет перемещать эти поддельные точки ближе к реальным данным, итеративно все ближе и ближе, так что эти поддельные примеры будут почти идентичны, следуя распределению реальных данных.

Intuition behind GANs

Generator tries to improve its imitation of the data.



● Real data

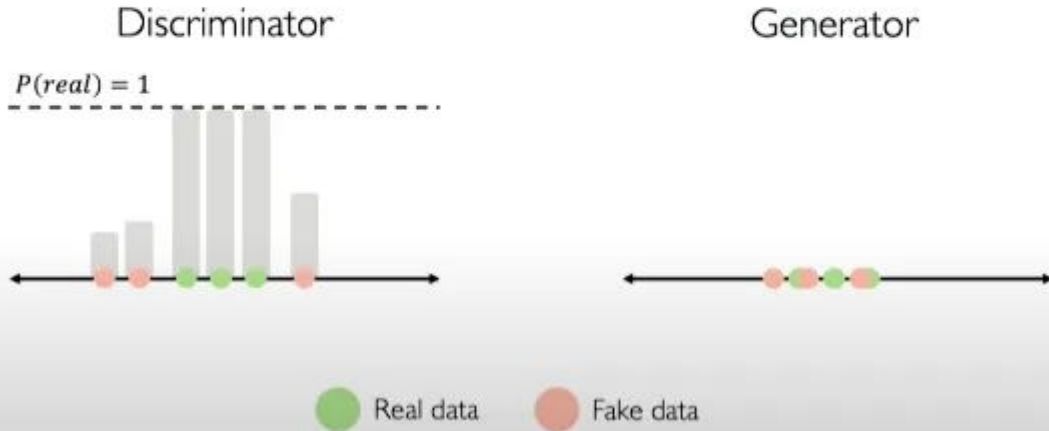
● Fake data

В какой-то момент дискриминатору станет очень сложно отличить правду от лжи, в то время как генератор будет продолжать пытаться улучшить качество генерируемых им образцов, чтобы обмануть дискриминатор.

Intuition behind GANs

Discriminator tries to identify real data from fakes created by the generator.

Generator tries to create imitations of data to trick the discriminator.



Training GANs

Обучение GAN можно выразить через оптимизационную задачу:

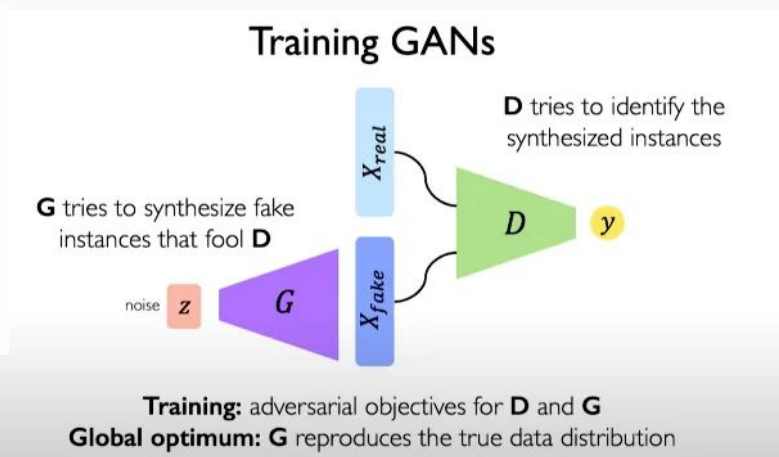
$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Эта функция стоимости представляет собой игру между двумя игроками:

- Дискриминатор D стремится максимизировать вероятность правильной классификации.
- Генератор G пытается минимизировать эту же величину, то есть хочет максимизировать вероятность того, что дискриминатор ошибётся.

В равновесии генератор производит образцы, которые дискриминатор не может отличить от реальных, т.е.

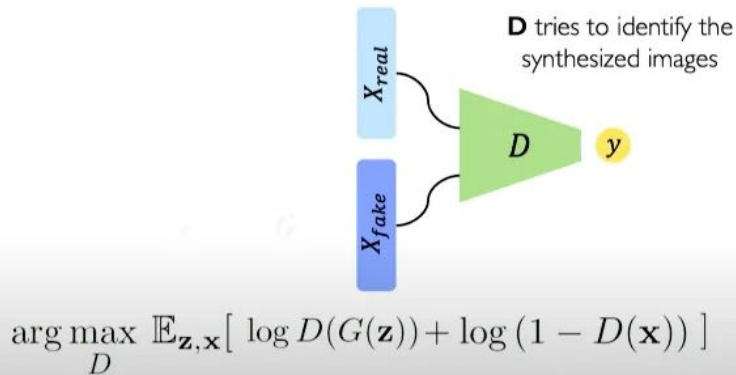
$$p_g(x) = p_{\text{data}}(x).$$



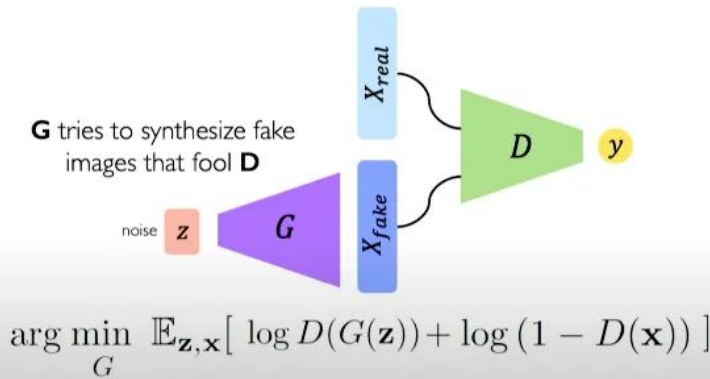
Функция потерь для обучения GAN направлена на максимизацию вероятности того, что дискриминатор определит поддельные данные как поддельные, а реальные данные как реальные. Это то, что называется кросс-энтропийной потерей между истинным распределением и распределением, сгенерированным сетью-генератором. Цель дискриминатора — максимизировать эту цель в целом.

Для генератора у нас тот же общий компонент, но теперь мы пытаемся минимизировать цель с точки зрения генератора. Поскольку генератор не может напрямую получить доступ к истинному распределению данных $D(x)$, его внимание сосредоточено на минимизации распределения и члена потерь $D(G(z))$, что минимизирует вероятность того, что его сгенерированные данные будут идентифицированы как поддельные.

Training GANs: loss function



Training GANs: loss function

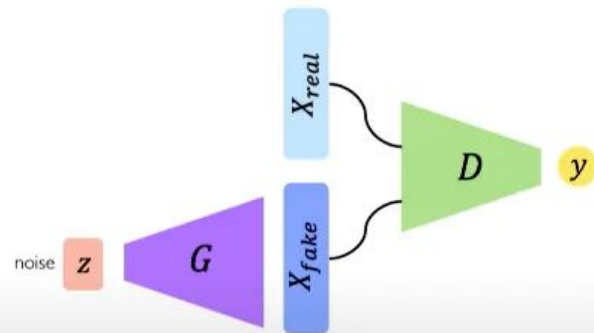


После того, как сеть генератора полностью обучена, сделайте из нее выборку, чтобы создать новые экземпляры данных, которые никогда ранее не встречались.

Способ, которым генератор синтезирует эти новые экземпляры данных, идет от распределения совершенно случайного гауссовского шума и обучения функции, которая отображает преобразование от этого гауссовского шума к целевому распределению данных. Это приближение заключается в том, что генератор обучается в ходе самого обучения.

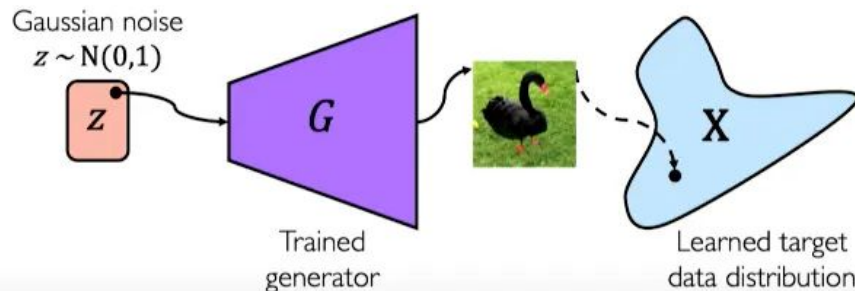
Одна точка в распределении шума приведет к одной точке в целевом распределении

Generating new data with GANs



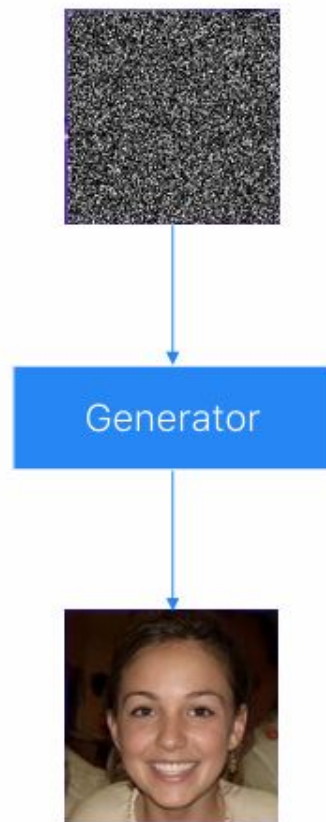
After training, use generator network to create **new data** that's never been seen before.

GANs are distribution transformers

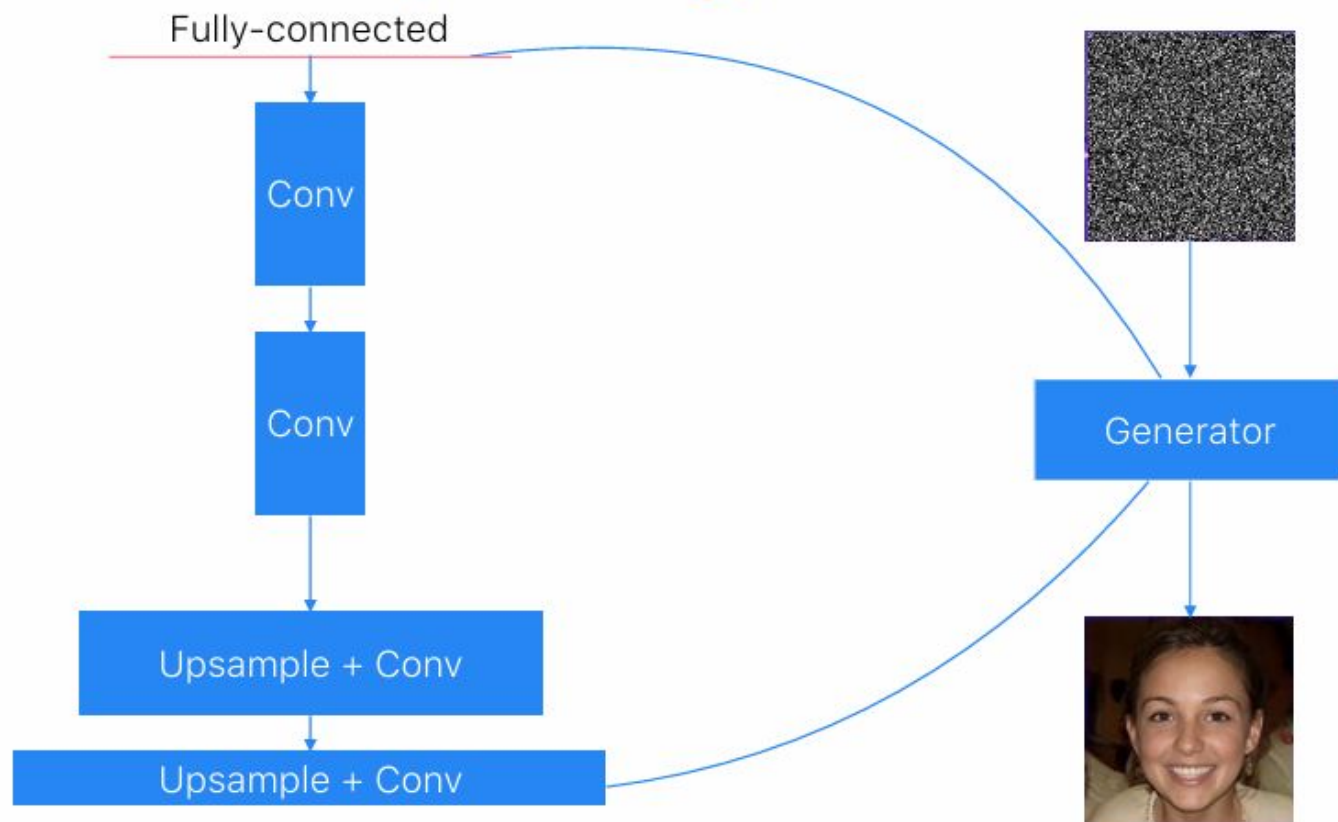


Классическая архитектура

Подаем на вход шум. На выходе получаем изображение из датасета.



Классическая архитектура



Виды генеративных сетей *

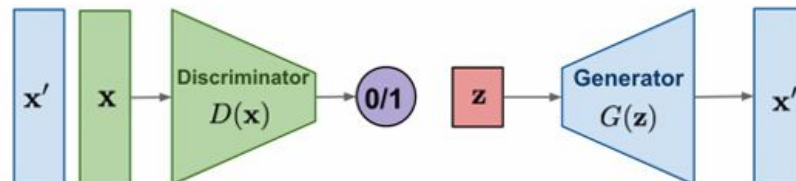
1. Вариационные автоэнкодеры (VAE)

2. Генеративные состязательные сети (GAN).

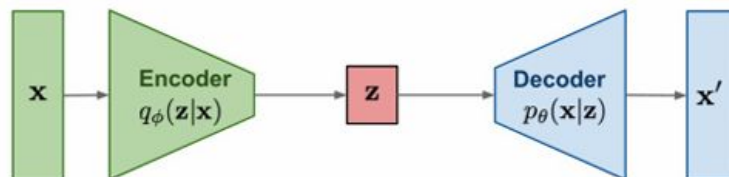
3. Flow-based models

4. Diffusion (недавняя тенденция)

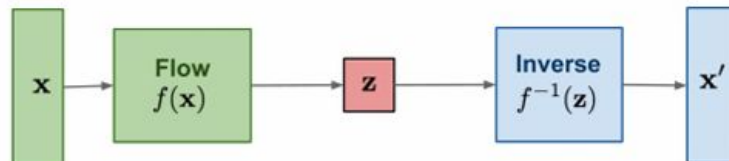
GAN: Adversarial training



VAE: maximize variational lower bound



Flow-based models:
Invertible transform of distributions



Diffusion models:
Gradually add Gaussian noise and then reverse

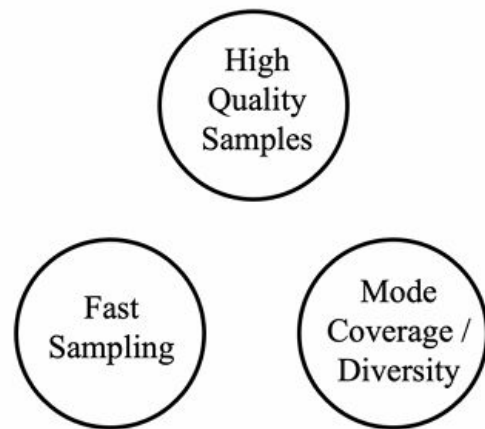


• Источник: <https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>

Какую модель выбрать? *

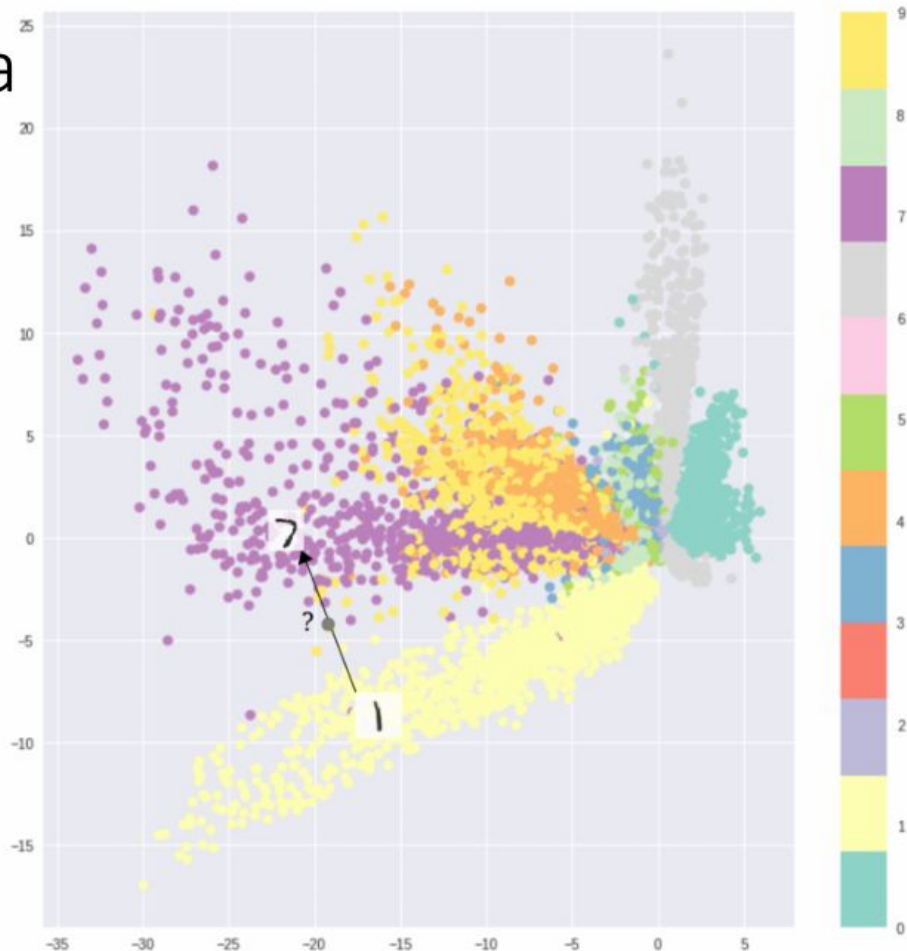
VAE	Поток	GAN	Диффузия
Высокая частота дискретизации. Генерация разнообразных образцов	Высокая частота дискретизации. Генерация разнообразных образцов	Высокая частота дискретизации. Высокое качество генерации образцов.	Высокое качество генерации образцов. Генерация разнообразных образцов
Низкое качество генерации выборки	Требуется специализированная архитектура, низкое качество генерации выборки	Нестабильное обучение, низкое разнообразие генерации выборки (коллапс режима)	Низкая частота дискретизации

The Generative Learning Trilemma

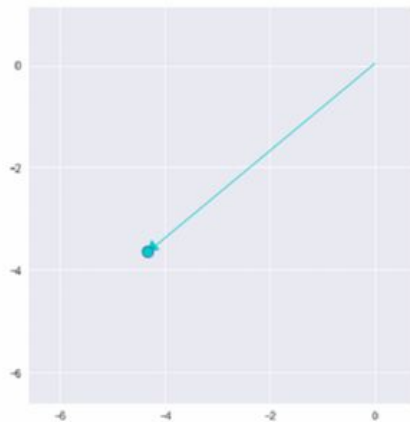


Эмбеддинг автоэнкодера

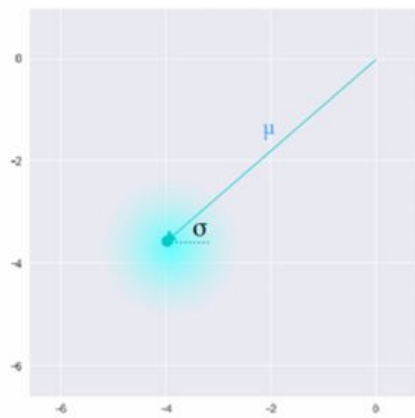
- Всё замечательно, если вам необходимо просто воспроизвести исходные изображения.
- Однако, если вы строите генеративную модель, ваша цель не простое дублирование изображений.
- Вы хотите получить случайное или видоизмененное изображение, восстановленное из непрерывного скрытого пространства.
- Если пространство имеет разрывы, то декодер выдаст нереалистичное изображение



Эмбеddинг VAE



Standard Autoencoder
(direct encoding coordinates)



Variational Autoencoder
(μ and σ initialize a probability distribution)

- Если вы хотите восстановить входные данные, вы просто выбираете подходящее распределение и посылаете его в декодер

